Drugi probni kolokvij

Web programiranje

Napomena: ~~Na kolokviju je dopušteno koristiti materijale s nastave te sve dostupne izvore s neta. Nije dopušteno međusobno komunicirati. Rješenja se provjeravaju programom za otkrivanje plagijata. Prije komprimiranja rješenja uklonite sve mape~~ `node_modules` ~~*(1 bod)*. Komprimirano rješenje~~ `prezime_ime_kolokvij_2.zip` ~~pošaljite nastavniku na Teams.~~

Pri uspješnoj implementaciji oba zadatka, rješenje se treba ponašati kao na `capture.mp4`.

Zadatak 1 ~~(28 bodova)~~

Raspakirajte `template.zip`, mapu preimenujte u `prezime_ime_kolokvij_2`, pokrenite MongoDB poslužitelj, instalirajte pakete za poslužitelj i klijent iz predloška te ih pokrenite[1].

U zaglavlju datoteke `auth.service.ts` definirani su složeni tipovi koji deklariraju podatke potrebne za prijavu (*eng. sign in*), podatke potrebne za registraciju (*eng. sign up*) te podatke unutar payload-a JWT tokena.

Na klijentu, unutar `AuthService` implementirajte metode `save_token`, `get_token`, `logout` i `token_payload` koje će spremati JWT token u `localStorage`, dohvaćati token iz `localStorage`-a, obrisati token iz `localStorage`-a te dohvaćati payload iz tokena ukoliko token postoji *(2 boda + 2 boda + 2 boda + 2 boda)*.

U istome servisu implementirajte metode `sign_in(credentials: SignInCredentials)` te `sign_up(credentials: SignUpCredentials)` koje prihvaćaju podatke prikupljene s odgovarajućih formi te ih putem HTTP zahtjeva šalju prema poslužitelju, a pri uspješnom odgovoru s poslužitelja, obje ove metode pohranjuju token *(4 boda + 4 boda)*.

U `SignUpComponent` i `SignInComponent` implementirajte odgovarajuće funkcionalnosti za podnošenje formi *(3 boda za formu + 3 boda za podnošenje forme + 3 boda za formu + 3 boda za podnošenje forme)* tako da se odgovarajući podaci pošalju prema poslužitelju gdje je pri uspješnom odgovoru s poslužitelja potrebno ispisati odgovor te preusmjeriti prikaz na `ProfileComponent`. Pri neuspješnom odgovoru, npr. zbog slanja neispravnih podataka za prijavu, potrebno je ostati na prikazu iste komponente.

[1] Klijent je dovoljno pokrenuti s `ng serve` budući da je datoteka `proxy.conf.json` koja preusmjerava HTTP pozive već uključena unutar konfiguracijske datoteke `angular.json`.

Zadatak 2 ~~(31 bod)~~

Na poslužitelju implementirajte HTTP POST rutu za registraciju s putanjom `"/api/sign-up"`. Na odgovarajuć način pohranite podatke za korisnika u bazu podataka *(3 boda)*. Zatim stvorite token koji će trajati 24 sata pri čemu ćete u payload tokena postaviti, uz ostale potrebne atribute, korisničko ime i email korisnika te token pošaljite prema klijentu *(3 boda)*.

Na poslužitelju implementirajte HTTP POST rutu za prijavu s putanjom `"/api/sign-in"`. Na osnovu podataka poslanih s klijenta, pokušajte pronaći korisnika u bazi podataka *(1 bod)*. Ukoliko ne postoji, pošaljite smisleni odgovor *(1 bod)*. U suprotnom provjerite je li lozinka valjana *(1 bod)*. Ukoliko je, stvorite i klijentu pošaljite token kao i pri registraciji *(3 boda)*, a ukoliko nije, pošaljite smisleni odgovor *(1 bod)*.

U prikazu `ProfileComponent` prikažite podatke dobivene iz payload-a tokena *(2 boda + 2 boda)*. Pritiskom na gumb „Logout“ potrebno je ukloniti token te se preusmjeriti na stranicu za prijavu *(2 boda + 2 boda)*.

Pri učitavanju klijenta, ukoliko token postoji (što podrazumijeva da iz payload-a možemo iščitati korisničke podatke) te inicijalni prikaz nije prikaz `ProfileComponent`, potrebno je preusmjeriti se na taj prikaz *(2 boda)*. Ukoliko je inicijalni prikaz prikaz `ProfileComponent`, a token ne postoji, potrebno je preusmjeriti prikaz na prikaz `SignInComponent` *(2 boda)*.

U navigacijskoj traci, ukoliko token ne postoji, potrebno je prikazati linkove do prikaza formi za prijavu i registraciju *(2 boda)*. Dodatno, u ovisnosti o tome koji je prikaz učitan, potrebno je podcrtati njegov link *(2 boda)*. Ukoliko token postoji, potrebno je ispisati poruku „Hello {{ username }}“ pri čemu je korisničko ime iščitano iz payload-a tokena *(2 boda)*.